

Course: Full Stack Web Development

Course Code: 23CM4121

Task No.: 3

Student Name: M. Vanisree

Roll No.: A24126552125

Date: 25-08-2026

Title

Write a Program Using JavaScript (Arrays, Functions)

Aim

To design and develop a Contact List application using JavaScript arrays and functions, which allows the user to add, display, search, and delete contacts dynamically.

OBJECTIVE

The objectives of this task are:

- To understand the use of JavaScript arrays and objects.
 - To create and use JavaScript functions.
 - To store multiple contacts using an array.
 - To add new contacts dynamically.
 - To display contacts on a webpage using DOM manipulation.
 - To delete contacts using the splice() method.
 - To search contacts using includes().
 - To implement case-insensitive searching using toLowerCase().
 - To handle user actions using addEventListener().
 - To dynamically display the total number of contacts.
-

Theory

JavaScript is a scripting language used to make web pages interactive and dynamic. In this task, a **Contact List application** is developed using HTML, CSS, and JavaScript.

An **array** is used to store multiple contact objects. Each contact object contains the contact's name and phone number. JavaScript functions are used to perform different operations such as adding, displaying, deleting, and searching contacts.

The application provides input fields for entering a contact's name and phone number. When the **Add Contact** button is clicked, the contact is stored in the array using the `push()` method and displayed on the webpage.

Each contact contains a **Delete** button. When the Delete button is clicked, the corresponding contact is removed from the array using the `splice()` method.

A search box is provided to filter contacts by name while the user types. The `toLowerCase()` and `includes()` methods are used to perform case-insensitive searching.

The total number of contacts is also updated dynamically whenever a contact is added or deleted.

Concepts Used

Concept	Purpose
Array	Stores multiple contacts
Object	Stores the name and phone number of a contact
Function	Performs specific operations
<code>push()</code>	Adds a contact to the array
<code>splice()</code>	Deletes a contact from the array
<code>forEach()</code>	Iterates through the contacts
DOM Manipulation	Dynamically updates webpage content
<code>getElementById()</code>	Accesses HTML elements
<code>innerHTML</code>	Displays dynamic HTML content
<code>addEventListener()</code>	Detects user actions
<code>includes()</code>	Searches for a contact name
<code>toLowerCase()</code>	Makes searching case-insensitive
Template Literals	Creates dynamic HTML content

ALGORITHM / PROCEDURE

1. Create an HTML file containing input fields for the contact name and phone number.
2. Create an **Add Contact** button.
3. Create a search input field for searching contacts.

4. Create a container to display the contact list.
5. Create a section to display the total number of contacts.
6. Create a CSS file to style the webpage.
7. Create a JavaScript array named contacts.
8. Create the addContact() function.
9. Retrieve the name and phone number entered by the user.
10. Validate whether both fields contain values.
11. Create a contact object containing the name and phone number.
12. Add the contact object to the array using push().
13. Call the displayContacts() function.
14. Display all contacts using the forEach() method.
15. Create a Delete button for each contact.
16. Create the deleteContact() function to remove a contact using splice().
17. Update the total number of contacts after adding or deleting a contact.
18. Create the searchContact() function.
19. Retrieve the search value and convert it to lowercase.
20. Use includes() to find matching contact names.
21. Display only the matching contacts.
22. Attach the search function to the input event using addEventListener().
23. Execute the application in a web browser.
24. Perform multiple test cases and verify the expected and actual results.

Code

HTML Code

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Contact List</title>
  <link rel="stylesheet" href="./style.css">
</head>

<body>

  <h1>Contact List</h1>

  <div class="info">

    <input
      type="text"
      id="name"
      placeholder="Enter contact name"
    >

    <input
      type="text"
      id="phone"
      placeholder="Enter phone number"
    >

    <button onclick="addContact()">Add Contact</button>

  </div>

  <input
    type="search"
    placeholder="Search the contact"
    id="search"
  >

  <div id="contactList"></div>

  <p id="totalContacts">Total Contacts: 0</p>

  <script src="./script.js"></script>

</body>
</html>

```

CSS Code:

```

body {
  background-color: white;
  font-family: Arial, sans-serif;
  margin: 0;
  padding: 20px;
}

/* Heading */
h1 {
  background-color: cadetblue;
  color: aliceblue;
  width: 250px;
  margin: 20px auto;
  padding: 10px;
  text-align: center;
  border-radius: 12px;
  box-shadow: 5px 5px 10px rgba(0, 0, 0, 0.15);
}

/* Add Contact Section */
.info {
  background-color: rgb(211, 96, 96);
  margin: 20px auto;
  padding: 20px;
  border-radius: 12px;
  width: 600px;
  box-shadow: 8 12px 40px rgba(0, 0, 0, 0.15);
}

/* Input boxes */
input {
  display: block;
  width: 90%;
  margin: 10px auto;
  padding: 10px;
  border: none;
  border-radius: 8px;
  box-sizing: border-box;
  outline: none;
}

/* Buttons */
button {
  border: none;
  padding: 10px 20px;
  background-color: cadetblue;
  color: aliceblue;
  border-radius: 8px;
  display: block;
  margin: 10px auto;
  cursor: pointer;
}

button:hover {
  background-color: rgb(61, 130, 132);
}

/* Search */
#search {
  display: block;

```

```

/* Search */
#search {
  display: block;
  width: 60%;
  margin: 20px auto;
  padding: 10px;
  border: none;
  background-color: rgb(211, 96, 96);
  color: white;
  border-radius: 12px;
  box-shadow: 5px 5px 10px rgba(0, 0, 0, 0.15);
  outline: none;
}

#search::placeholder {
  color: bisque;
}

/* Contact List */
#contactList {
  width: 60%;
  margin: 20px auto;
}

/* Contact Cards */
#contactList .info {
  background-color: rgb(211, 96, 96);
  color: bisque;
  width: auto;
  margin: 10px 0;
  padding: 20px;
  border-radius: 12px;
  box-shadow: 8 8px 20px rgba(0, 0, 0, 0.15);
}

/* Contact name */
.info h2 {
  margin-top: 0;
  text-align: center;
}

/* Contact phone */
.info p {
  text-align: center;
}

/* Total contacts */
#totalContacts {
  text-align: center;
  font-size: 18px;
  font-weight: bold;
  margin-top: 20px;
}

```

Javascript Code:

```
let contacts = [];  
  
// Add Contact  
function addContact() {  
    let name = document.getElementById("name").value;  
    let phone = document.getElementById("phone").value;  
  
    if (name === "" || phone === "") {  
        alert("Please enter name and phone number");  
        return;  
    }  
  
    let contact = {  
        name: name,  
        phone: phone  
    };  
  
    contacts.push(contact);  
  
    document.getElementById("name").value = "";  
    document.getElementById("phone").value = "";  
  
    displayContacts();  
}  
  
// Display Contacts  
function displayContacts() {  
    let contactlist = document.getElementById("contactlist");  
  
    contactlist.innerHTML = "";  
  
    contacts.forEach(function(contact, index) {  
        contactlist.innerHTML += `  
        <div class="info">  
  
            <h2>${contact.name}</h2>  
  
            <p>${contact.phone}</p>  
  
            <button onclick="deleteContact(${index})">  
                Delete  
            </button>  
  
        </div>  
        `;  
    });  
  
    document.getElementById("totalContacts").innerText =  
        "Total Contacts: " + contacts.length;  
}
```

```
document.getElementById("totalContacts").innerText =  
    "Total Contacts: " + contacts.length;  
}  
  
// Delete Contact  
function deleteContact(index) {  
    contacts.splice(index, 1);  
  
    displayContacts();  
}  
  
// Search Contact  
function searchContact() {  
    let searchValue = document.  
        .getElementById("search")  
        .value  
        .toLowerCase();  
  
    let contactlist = document.getElementById("contactlist");  
  
    contactlist.innerHTML = "";  
  
    contacts.forEach(function(contact, index) {  
        if (contact.name.toLowerCase().includes(searchValue)) {  
            contactlist.innerHTML += `  
            <div class="info">  
  
                <h2>${contact.name}</h2>  
  
                <p>${contact.phone}</p>  
  
                <button onclick="deleteContact(${index})">  
                    Delete  
                </button>  
  
            </div>  
            `;  
        }  
    });  
}  
  
// Search while typing  
document.  
    .getElementById("search")  
    .addEventListener("input", searchContact);
```

CODE EXPLANATION

Code	Explanation
let contacts = []	Creates an empty array to store contact details.
function addContact()	Defines a function to add a new contact.
getElementById()	Selects an HTML element using its ID.

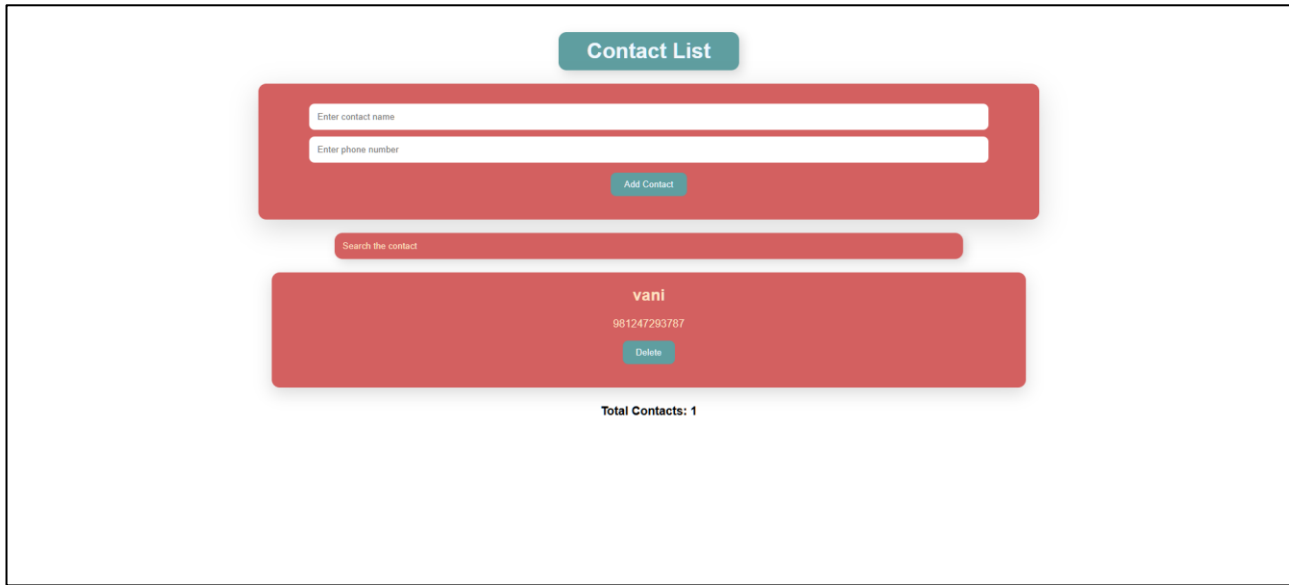
<code>.value</code>	Retrieves the value entered in an input field.
<code>`if (name === ""</code>	
<code>alert()</code>	Displays a warning message when required fields are empty.
<code>let contact = {}</code>	Creates an object containing the contact details.
<code>contacts.push(contact)</code>	Adds the contact object to the contacts array.
<code>displayContacts()</code>	Displays all stored contacts on the webpage.
<code>forEach()</code>	Iterates through each contact in the array.
<code>innerHTML = ""</code>	Clears the existing contact list before displaying updated content.
<code>innerHTML +=</code>	Adds dynamically generated contact information to the webpage.
template literals	Creates dynamic HTML content using backticks and <code>\${}</code> .
<code>deleteContact(index)</code>	Defines a function to delete a selected contact.
<code>splice(index, 1)</code>	Removes one contact from the array at the specified index.
<code>searchContact()</code>	Defines a function to search and filter contacts.
<code>toLowerCase()</code>	Converts text to lowercase for case-insensitive searching.
<code>includes(searchValue)</code>	Checks whether the contact name contains the search text.
<code>addEventListener("input", searchContact)</code>	Calls the search function whenever the user types in the search box.

contacts.length	Returns the total number of contacts stored in the array.
innerText	Updates the displayed total number of contacts.

TEST CASES AND EXECUTION DATA

Test Case	Component Tested	Action / Input	Expected Result	Actual Result	Status
TC-01	Add Contact	Enter name "Rahul" and phone "9876543210", then click Add Contact	Contact should be added and displayed	Contact added successfully	Pass
TC-02	Input Validation	Leave the name or phone field empty and click Add Contact	Alert message should be displayed	Alert displayed successfully	Pass
TC-03	Search Contact	Enter a contact name in the search box	Matching contact should be displayed	Matching contact displayed successfully	Pass
TC-04	Delete Contact	Click the Delete button of a contact	Selected contact should be removed	Contact deleted successfully	Pass
TC-05	Contact Count	Add or delete contacts	Total contact count should update dynamically	Contact count updated correctly	Pass

Output :



The screenshot displays a web application titled "Contact List". It features a red-themed interface. At the top, there is a teal button labeled "Contact List". Below this, a large red container holds two input fields: "Enter contact name" and "Enter phone number", each with a corresponding teal "Add Contact" button. A search bar labeled "Search the contact" is positioned below the input fields. The main display area shows a single contact entry for "vani" with the phone number "981247293787" and a teal "Delete" button. At the bottom, it indicates "Total Contacts: 1".

RESULT

Thus, a **Contact List application** was successfully developed using **JavaScript arrays, objects, functions, and DOM manipulation**. The application successfully performs contact addition, display, searching, deletion, and dynamic contact counting. The program was tested using multiple test cases and the expected results were verified successfully.